ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. І. ФРАНКА · ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

Unimap 2.0: розширення функціоналу та перехід на архітектуру Porto SAP

Продовження розробки інтерактивної картографічної системи для навігації університетом

Кафедра інформаційних систем

Виконав: студент групи ПМІ-42 **Зозуля О. Ю.**

Керівник: **Козій І. Я.**

Етап 1 — виконано

Етап 2 — розширення та рефакторинг

ЕТАП 1 · ПІДСУМОК

Що вже зроблено в дипломній роботі

Перша версія Unimap — повнофункціональний веб-застосунок навігації приміщеннями з пошуком аудиторій і найкоротшими маршрутами між поверхами.

Цілі етапу 1 ВИКОНАНО

- ✓ Проаналізовано наявні рішення (паперові плани, Google Maps, MazeMap)
- ✓ Обґрунтовано вибір технологій (TypeScript наскрізно)
- ✓ Спроєктовано архітектуру й графову модель поверхів
- ✓ Реалізовано серверну, клієнтську та адмін-частини
- ✓ Виконано автоматизоване та ручне тестування

3
застосунки: api, web, admin

11
доменних модулів бекенду

A*
маршрути між поверхами

3
ролі: guest, user, admin

Жодне з проаналізованих рішень не поєднувало пошук, маршрути між поверхами, контроль даних ЗВО та безкоштовність — Unimap закрив цю прогалину.

ЕТАП 1 · ТЕХНОЛОГІЇ

Технологічний стек, що вже працює

Монорепозиторій на npm workspaces, єдина мова TypeScript (strict) на сервері та клієнті.

Бекенд `apps/api`

- NestJS 11 + Fastify
- REST + GraphQL (Mercurius, DataLoader)
- Zod-валідація (`nestjs-zod`), конфіг fail-fast
- JWT + cookie-сесії, Throttler

Дані та інфраструктура

- PostgreSQL 16 + Drizzle ORM
- Redis 7 + BullMQ (черги листів)
- MinIO (S3) — SVG-джерела карт
- Mailpit, Docker Compose

Фронтенд `apps/web` , `apps/admin`

- Next.js 15 + React 19 (SSR)
- TanStack Query, Zustand
- next-intl (локалізація)
- Tailwind 4, react-zoom-pan-pinch

Продуктивність
Fastify, SSR, кеш графів

Типобезпека
наскрізно від БД до UI

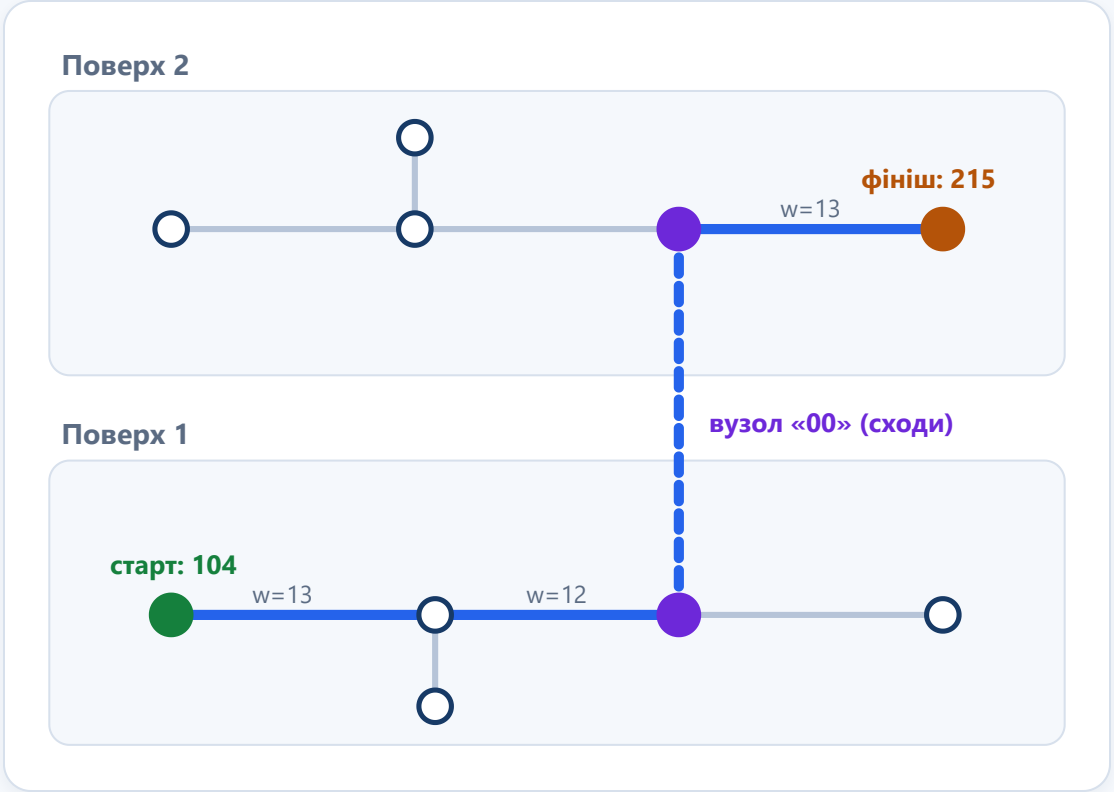
Якість
Jest, e2e, ESLint 9, Prettier

Модульність
DDD-lite, SOLID

План поверху як зважений граф + A*

- ✓ Вхід — розмічений SVG: `room` · `node` · `edge`
- ✓ Вага ребра = довжина відрізка на плані
- ✓ Вузли з кодом «00» з'єднують поверхи (сходи, ліфти)
- ✓ BFS-перевірка зв'язності перед публікацією корпусу
- ✓ A* з евклідовою евристикою — допустима, тому шлях оптимальний
- ✓ Пріоритетна черга на бінарній купі, об'єднаний граф корпусу

> Розбір SVG виконує `SvgFloorParser` через реєстр парсерів — формат джерел можна розширювати.



ЕТАП 1 · ФУНКЦІОНАЛ КОРИСТУВАЧА

Що вміє система для студентів і гостей ВИКОНАНО

Пошук
Швидкий пошук аудиторій за ключовими словами з підказками

Фільтр
Множинний фільтр приміщень за категоріями

Опис приміщення
Повна картка обраного приміщення на карті

Маршрут
Найкоротший шлях між двома приміщеннями, зокрема між поверхами

Історія
Історія пошуку приміщень і маршрутів, повтор маршруту

Збережене
Особисті збережені приміщення та маршрути

Події
Перегляд глобальних подій і створення особистих

Акаунт
Реєстрація з підтвердженням e-mail, сесії, локалізація інтерфейсу

ЕТАП 1 · АДМІНІСТРУВАННЯ

Адмін-панель `apps/admin` ВИКОНАНО

Корпуси

Створення, редагування, публікація (`draft` → `live`) і видалення корпусів

Завантаження карт

Імпорт SVG-планів поверхів із перевіркою розмітки, версії, активація версії

Приміщення

Редагування метаданих приміщень після завантаження джерела

Події університету

Створення та планування глобальних подій на карті

Користувачі

Перегляд акаунтів, керування ролями `guest` / `user` / `admin` , видалення

Аудит і журнали

Аудит дій адміністраторів, збір клієнтських логів (`client-logs`)

Поточна архітектура бекенду: DDD-lite

```
apps/api/src/
├─ core/      # інфраструктура
│  ├─ auth  config  database
│  ├─ graphql  storage  audit
│  └─ filters  pipes  interceptors
├─ shared/    # спільні утиліти
└─ modules/   # 11 модулів
   ├─ auth  users  maps  rooms
   ├─ wayfinding  history  saved
   ├─ events  notifications
   └─ health  client-logs
      └─ controllers/ services/
         repositories/ dtos/ types/
```

Що працює добре

Feature-first модулі, тонкі контролери, Zod-DTO, репозиторії над Drizzle, ізольована інфраструктура в `core/`.

Обмеження, що заважають рости

ПРОБЛЕМА

- Сервіси розростаються: `maps-admin.service.ts` поєднує завантаження, валідацію, версіонування й публікацію
- Оркестрація й атомарні операції змішані в одному класі
- Пряма залежність між модулями: `ShortestPathService` імпортує `HistoryService` і `RoomsRepository`
- Межі модулів не контролюються — складно виділити мікросервіс

02

Новий функціонал

Розширення можливостей застосунку: від доступності й розкладу до аналітики та редактора карт.

ЕТАП 2 · ДОРОЖНЯ КАРТА

Пропоновані можливості та пріоритети

Можливість	Цінність для користувача	Пріоритет	Новий контейнер
Доступні маршрути	Маршрути без сходів, покрокові й голосові підказки	ВИСОКИЙ	`Navigation`
Розклад занять	«Моя наступна пара» — маршрут в один клік і нагадування	ВИСОКИЙ	`Schedule`
Зайнятість і бронювання	Вільні аудиторії в реальному часі, заявки на бронь	ВИСОКИЙ	`Booking`
QR-позиціонування, PWA	Точка старту з QR-коду, робота офлайн, встановлення на телефон	СЕРЕДНІЙ	`Positioning`
Кілька кампусів	Маршрути між корпусами по вулиці, підтримка кількох ЗВО	СЕРЕДНІЙ	`Campus`, `Tenant`
Аналітика	Теплові карти запитів і популярні маршрути для адміністрації	СЕРЕДНІЙ	`Analytics`
Редактор карт	Візуальне редагування графа без ручної SVG-розмітки	НИЖЧИЙ	`MapEditor`

ЕТАП 2 · НОВА МОЖЛИВІСТЬ НОВЕ

Доступні маршрути та покрокова навігація

Університет має бути зручним для всіх: людей на кріслах колісних, з дитячими візками, з порушеннями зору.

Профіль «без сходів»

- Тип вузла «00»: `stairs`, `elevator`, `ramp`
- A* з функцією вартості профілю: сходи заборонені або штрафуються
- Позначка недоступних ділянок

Покрокові підказки

- Генерація інструкцій з кутів між ребрами: «поверніть ліворуч», «підніміться на 3 поверх»
- Орієнтири: найближчі аудиторії
- Локалізація через next-intl

Голос і контраст

- Озвучення кроків (Web Speech API)
- Висококонтрастна тема, масштабування шрифтів
- Відповідність WCAG 2.2 AA

Технічно: `RouteProfile` передається у `FindPathAction`, а `RunAStarTask` отримує стратегію ваг ребер — алгоритм не змінюється, змінюється лише функція вартості.

ЕТАП 2 · НОВА МОЖЛИВІСТЬ НОВЕ

Розклад занять і «Моя наступна пара»

- **Імпорт розкладу** факультету з iCal / CSV або з API деканату; прив'язка пари до коду аудиторії
- **Підписка** студента на групу, викладача — на свої дисципліни
- **«Моя наступна пара»**: картка на головній сторінці та маршрут в один клік від поточної аудиторії
- **Нагадування** за N хвилин до пари: e-mail і Web Push через BullMQ (вже наявна черга)
- **Зміни в розкладі**: сповіщення про перенесення аудиторії

ImportScheduleAction

ParseIcalTask → MatchRoomsTask → UpsertLessonsTask

GetNextLessonAction

FindNextLessonTask → викликає FindPathAction контейнера Wayfinding

LessonReminderJob

Відкладене завдання BullMQ → SendNotificationTask

Новий контейнер `CampusLife/Schedule` використовує Wayfinding лише через його Action — без прямого доступу до чужих репозиторіїв.

ЕТАП 2 · НОВА МОЖЛИВІСТЬ НОВЕ

Зайнятість аудиторій і бронювання

Статус у реальному часі

Кожне приміщення на карті має колір: вільне, зайняте, скоро звільниться. Дані — з розкладу, бронювань і подій. Оновлення через WebSocket / GraphQL Subscriptions (Mercurius).

Пошук вільної аудиторії

Фільтр «вільна зараз / о 14:30», місткість, наявність проєктора чи комп'ютерів. Сортування за відстанню від користувача.

Заявки на бронювання

Студентські організації та викладачі подають заявку, адміністратор підтверджує в `apps/admin`. Перевірка конфліктів за інтервалами часу.

Заявка
CreateBookingAction

→

Перевірка конфлікту
CheckSlotConflictTask (PostgreSQL range + exclusion constraint)

→

Модерація
ApproveBookingAction (роль admin)

→

Подія
BookingApproved → оновлення карти й сповіщення

ЕТАП 2 · НОВА МОЖЛИВІСТЬ НОВЕ

QR-позиціонування та PWA

GPS не працює всередині будівель — точку старту задає сам користувач, відсканувавши QR-код біля входу, сходів чи аудиторії.

QR-маркери

- Генерація QR для будь-якого вузла графа в адмін-панелі, друк наборів наліпок у PDF
- Посилання формату `/b/{building}/n/{node}` — відкриває карту з уже заданим стартом
- Статистика сканувань — для аналітики потоків

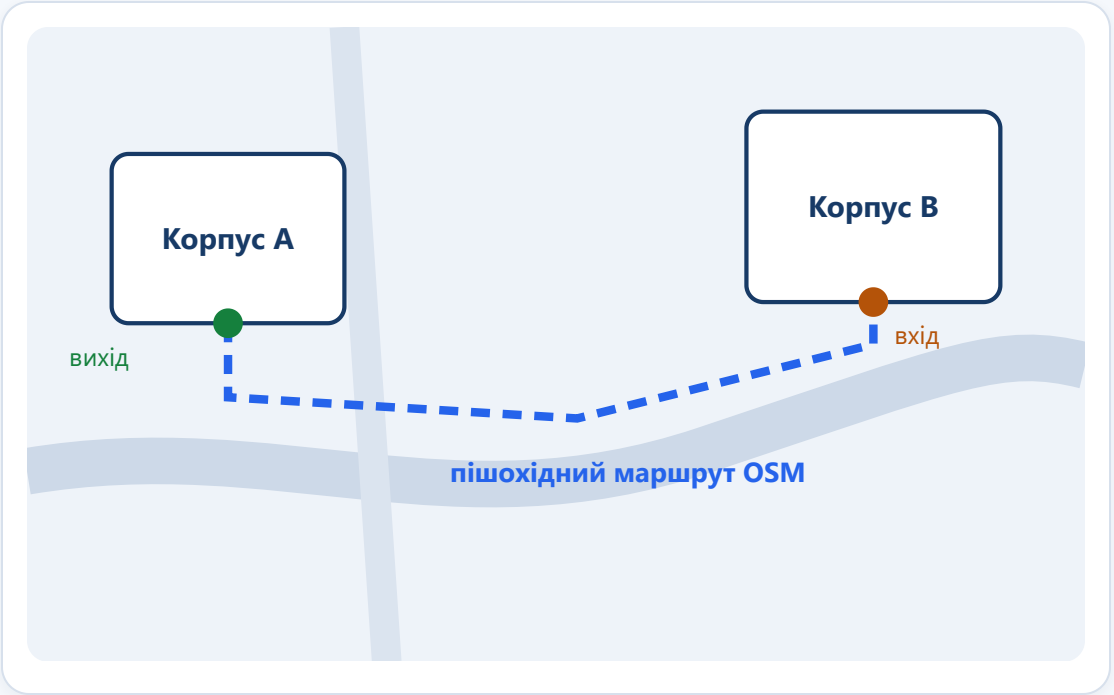
Progressive Web App

- Встановлення на головний екран телефону
- Service Worker кешує активні версії карт і графів — маршрут будується офлайн (A* на клієнті)
- Web Push для нагадувань про пари та події

ЕТАП 2 · НОВА МОЖЛИВІСТЬ НОВЕ

Кілька кампусів і зовнішня навігація

- **Кампус** як нова сутність над корпусами: координати входів корпусів на мапі міста
- **Маршрут «аудиторія → аудиторія» в різних корпусах**: внутрішній шлях до виходу + пішохідний маршрут OpenStreetMap (OSRM / GraphHopper) + шлях від входу
- **Мапа міста** з корпусами (Leaflet / MapLibre) на головній сторінці
- **Multi-tenant**: кілька університетів в одному розгортанні, ізоляція даних за `tenant_id`, власний брендинг



ЕТАП 2 · НОВІ МОЖЛИВОСТІ ДЛЯ АДМІНІСТРАЦІЇ НОВЕ

Аналітика та візуальний редактор карт

Дашборд аналітики

- Теплова карта пошукових запитів і побудованих маршрутів по поверхах
- Топ популярних приміщень і маршрутів, пікові години
- Запити без результату — підказка, які приміщення погано описані
- «Мертві зони» графа: вузли, через які шлях не проходить
- Збір подій асинхронно через BullMQ, агрегація у матеріалізованих поданнях PostgreSQL

Редактор карт у браузері

- Завантаження плану як підкладки, малювання вузлів і ребер мишею
- Автоматичний розрахунок ваг, прив'язка до сітки, підсвічування незв'язних компонент (BFS у реальному часі)
- Чернетка → перевірка → публікація, повна історія версій і відкат
- Експорт у той самий формат SVG — сумісність з етапом 1

03

Міграція на архітектуру Porto SAP

Структурований модульний підхід, що дозволяє рости від моноліту до мікросервісів без переписування бізнес-логіки.

PORTO SAP · ОСНОВИ PORTO

Що таке Porto SAP

Software Architectural Pattern, на якому побудовано фреймворк Apiato. Поєднує SOLID, DDD і MVC; ділить код на два шари.

Containers — бізнес-логіка

Незалежні модулі, згруповані в секції (Section). Кожен контейнер володіє своїми моделями, діями та інтерфейсами.

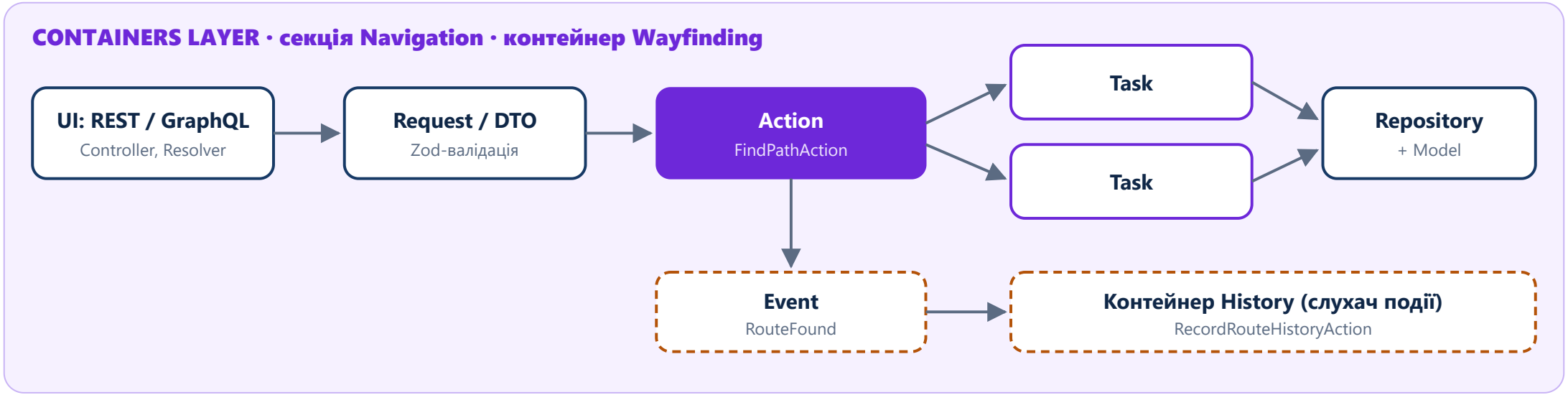
Ship — фундамент

Спільна інфраструктура, утиліти, конфігурація та базові класи, доступні всім контейнерам. Без бізнес-логіки.

Правила взаємодії компонентів

- **Route** → **Controller**: прийняти запит, валідувати Request, викликати рівно один Action
- **Action**: один сценарій використання, оркеструє Tasks
- **Task**: одна атомарна операція, перевикористовується різними Actions
- **Repository / Model**: доступ до даних тільки через Task
- Controller не містить логіки; Task не викликає Action
- Контейнери спілкуються лише через Actions/Tasks іншого контейнера або Events

Шари та потік запиту в Unimap



Нова структура `apps/api/src`

```
apps/api/src/
├─ ship/                  # було core/ + shared/
│  ├─ parents/  # BaseAction, BaseTask, BaseRepository
│  ├─ config/  database/  graphql/
│  ├─ auth/  storage/  queue/  mail/
│  └─ audit/  filters/  pipes/  interceptors/
└─ containers/
   ├─ identity/      Auth · User
   ├─ mapping/       Building · Floor · MapAsset · Room
   ├─ navigation/    Wayfinding · History · Saved
   ├─ campus-life/   Event · Notification
   │                 + Schedule · Booking (нові)
   └─ platform/      Health · ClientLog
                     + Analytics (новий)
```

```
containers/navigation/wayfinding/
├─ actions/      # сценарії
├─ tasks/        # атомарні операції
├─ data/
│  ├─ repositories/
│  └─ mappers/
├─ models/       # Drizzle-схеми
├─ ui/
│  ├─ api/controllers/
│  └─ graphql/resolvers/
├─ dtos/  events/  jobs/
├─ utils/        # BinaryMinHeap
├─ tests/  unit/ e2e/
└─ wayfinding.container.module.ts
```

Було → стало

Поточна структура (DDD-lite)	Porto SAP	Що змінюється
`core/*`, `shared/*`	`ship/*`	Перенесення без змін логіки
`modules/<feature>/`	`containers/<section>/<container>/`	Групування за секціями
`services/*.service.ts`	`actions/*.action.ts` + `tasks/*.task.ts`	Розбиття: сценарій і атомарні кроки
`controllers/`, GraphQL resolvers	`ui/api/controllers/`, `ui/graphql/`	Контролер викликає рівно один Action
`repositories/`, `mappers/`	`data/repositories/`, `data/mappers/`	Виклик лише з Tasks
`core/database/schema/*.ts`	`<container>/models/*.ts`	Модель належить своєму контейнеру
`notifications/processors`, `queues`	`<container>/jobs/`	Фонові завдання поруч із доменом
Прямі імпорти сервісів інших модулів	Публічні Actions або Events	Контрольовані межі контейнерів

Декомпозиція сценарію «Знайти маршрут»

Зараз: один сервіс робить усе

`ShortestPathService.find()` перевіряє вхідні дані, шукає кімнати через `RoomsRepository`, завантажує граф через `BuildingGraphLoaderService`, виконує A* і записує історію через `HistoryService` іншого модуля.

```
FindPathAction.run(dto)
  ├─ ResolveRoomNodesTask   // Mapping: room → node
  ├─ LoadBuildingGraphTask  // кеш об'єднаного графа
  ├─ RunAStarTask           // чиста функція + heap
  ├─ BuildPathStepsTask     // поверхи, відстань
  └─ emit RouteFoundEvent     // History слухає
```

```
// containers/navigation/wayfinding/actions
@Injectable()
export class FindPathAction extends BaseAction {
  constructor(
    private resolveNodes: ResolveRoomNodesTask,
    private loadGraph: LoadBuildingGraphTask,
    private aStar: RunAStarTask,
    private buildSteps: BuildPathStepsTask,
    private events: EventEmitter2,
  ) { super(); }

  async run(dto: FindPathDto, userId?: string) {
    const { from, to } = await this.resolveNodes
      .run(dto.buildingCode, dto.from, dto.to);
    const graph = await this.loadGraph.run(dto.buildingCode);
    const path = this.aStar.run(graph, from, to, dto.profile);
    const result = this.buildSteps.run(graph, path);
    if (dto.record !== false)
      this.events.emit('route.found', { userId, result });
    return result;
  }
}
```

Поетапна міграція без зупинки розробки

#	Етап	Роботи	Запобіжник
1	**Ship**	Перенести `core/` і `shared/` у `ship/`, додати `BaseAction`, `BaseTask`, оновити path-аліаси	typecheck + усі тести зелені
2	**Пілот: Wayfinding**	Розбити `ShortestPathService` на Action + Tasks, історію перевести на подію	e2e маршрутів без змін
3	**Mapping, Identity**	Розбити `maps-admin.service.ts`: Upload, Validate, Activate, Publish Actions	Стабільний REST/GraphQL-контракт
4	**Решта контейнерів**	Events, Notifications (+ jobs), Saved, History, Health, ClientLog	Покриття unit-тестами Tasks
5	**Захист меж**	ESLint `no-restricted-imports` між контейнерами, архітектурні тести (dependency-cruiser), шаблони генерації Action/Task	Перевірка в CI на кожен PR

Ризик — тимчасове співіснування двох структур. Рішення: міграція по одному контейнеру, старий модуль видаляється в тому самому PR, де з'являється новий.

PORTO SAP · РЕЗУЛЬТАТ PORTO

Що дає Porto для розвитку продукту

Масштабованість

Нові можливості етапу 2 (Schedule, Booking, Analytics, Campus) додаються як нові контейнери без зміни наявних.

Шлях до мікросервісів

Секція — готова межа сервісу: наприклад, `navigation` з A* можна винести в окремий сервіс під навантаженням.

Перевикористання

Один Task (`LoadBuildingGraphTask`) працює в маршрутах, пошуку вільних аудиторій і в офлайн-експорті графа для PWA.

Тестованість

Task — мала атомарна одиниця з явними залежностями; unit-тести без підняття всього модуля.

Читабельність

Назва Action описує сценарій: `PublishBuildingAction` , `CreateBookingAction` — структура коду відповідає вимогам.

Командна робота

Чіткі правила й межі — менше конфліктів, простіший онбординг нових розробників.

ПІДСУМКИ

Висновки та план робіт

Виконано

Етап 1: Unimap

Графова модель, A*, пошук, ролі, адмін-панель

Місяць 1

Ship + пілот

Перенесення інфраструктури, контейнер Wayfinding

Місяці 2–3

Повна міграція

Усі контейнери, захист меж у CI

Місяці 3–5

Пріоритетні фічі

Доступні маршрути, розклад, бронювання

Місяці 5–7

Розширення

QR + PWA, кампуси, аналітика, редактор карт

ВИКОНАНО

Повнофункціональний веб-застосунок навігації приміщеннями з графовою моделлю та маршрутами між поверхами.

НОВЕ

Сім напрямів розширення, що роблять Unimap щоденним інструментом студента й адміністрації.

PORTO

Перехід на Porto SAP усуває обмеження DDD-lite і готує систему до росту й мікросервісів.

Дякую за увагу!

GitHub: **github.com/juis6world/unimap**

Porto SAP: apiato.io/docs/architecture-concepts/porto